Homework 1 Report: Image Processing

1. Problem 1: Matrix

1.1 Algorithm

The input data is organized as a matrix X of size N by D, where N is the number of data samples and D is the number of features per sample. Each row of X represents one data sample. The objective is to compute a matrix Z of size N by N, where each entry Z(i, j) represents the Euclidean distance between sample i and sample j.

In the first implementation, a two-level nested loop is used. For every pair of samples (i, j), the difference between the two row vectors is computed. The Euclidean distance is obtained by taking the square root of the dot product of this difference vector with itself. This process is repeated for all N by N pairs, and the results are stored in the matrix Z.

In the second implementation, the same computation is performed without explicit loops. Instead, matrix operations are applied. First, the matrix multiplication between X and its transpose is computed to obtain all pairwise inner products. Then, the squared norm of each row is computed. Using these values, the squared distance between every pair of samples is calculated using the identity that the squared distance equals the squared norm of the first sample minus twice the inner product plus the squared norm of the second sample. The square root is applied element-wise to obtain the distance matrix Z. This relies on optimized matrix operations.

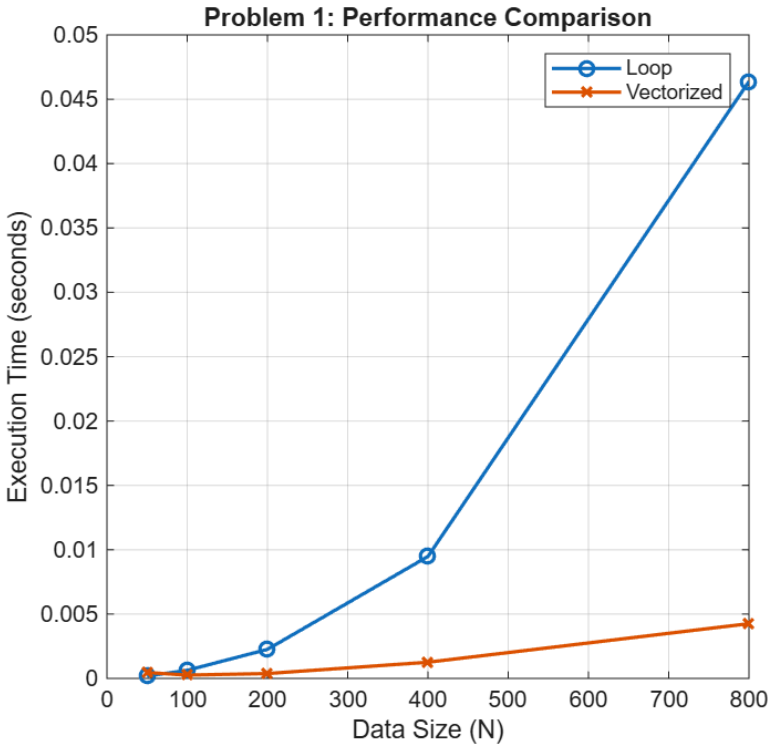
1.2 Result

In this experiment, the control variable is N, the number of data samples. The feature dimension D is held constant at 50 while N is varied. By increasing N and measuring execution time, we analyze how the computational cost scales with the number of data points.

Random matrices were generated with different values of N while keeping D fixed at 50. The execution time of both implementations was measured for each value of N, and the results were plotted with data size on the x-axis and execution time on the y-axis.

From the performance plot, execution time increases as N becomes larger because distances must be computed for all pairs of samples. For small data sizes, the execution times are comparable and the loop implementation may be slightly faster due to lower setup overhead. As N increases, the vectorized implementation becomes significantly faster than the nested loop implementation, and the performance gap grows with problem size.

Although both methods have the same theoretical time complexity, the vectorized version performs substantially better in practice. The nested loop implementation involves repeated interpreted operations in MATLAB, which introduce overhead. In contrast, the vectorized implementation leverages optimized low-level linear algebra routines, resulting in improved runtime performance.




```
Performance Results:
--------------------------------------------
N      Loop Time (s)      Vectorized Time (s)
--------------------------------------------
50     0.000185           0.000460
100    0.000630           0.000259
200    0.002258           0.000380
400    0.009476           0.001259
800    0.046383           0.004254
```

The output of both implementations is the pairwise distance matrix Z. This matrix is symmetric because the distance between sample i and sample j equals the distance between sample j and sample i. All diagonal elements are zero since the distance between a sample and itself is zero. In addition, all entries are non-negative. Both implementations produce identical results up to small numerical precision differences, confirming correctness. The results show that vectorization significantly improves performance in MATLAB.

2. Problem 2: Double Exposure

2.1 Algorithm

The objective of this problem is to combine two images to create a double exposure effect. Both images are first converted to double precision to prevent overflow during addition, since image data stored as uint8 may exceed the valid intensity range when pixel values are added directly.

Next, the dimensions of both images are obtained. The output image size is determined by taking the maximum height and maximum width of the two input images. A zero matrix of this size is initialized. Each image is then added to the output matrix at its corresponding pixel locations. To prevent saturation and maintain balanced brightness, the resulting pixel values are averaged by dividing by two. Finally, the output is converted back to uint8 format so that it can be displayed properly.

2.2 Result

Input image 1 (Everest_expedition.jpg):
Resolution: 800 × 538 pixels



Input image 2 (Everest_kalapatthar.jpg):

Resolution: 600 × 450 pixels



Blended Output Image (Double Exposure Result):
Resolution: 800 × 538 pixels



The output image is a blended composition that combines visual information from both input images. In overlapping regions, pixel values are averaged, creating a semi-transparent double exposure effect where features from both images are visible simultaneously. In regions where only one image contributes, the output reflects that image while maintaining consistent intensity scaling. The averaging step ensures that no pixel values exceed the valid range, preventing clipping or saturation artifacts.

To verify correctness, the output image dimensions were checked to confirm they equal max(height1, height2) by max(width1, width2), satisfying the problem constraint. Pixel values were examined before converting back to uint8 to ensure that no value exceeds 255, confirming that image saturation is avoided. Additionally, visual inspection verified that features from both input images, such as the tents and the mountain peak, appear together in the blended result.

3. Resources that Helped Me

MATLAB Documentation:

1. MATLAB scripts and functions
   https://www.mathworks.com/help/matlab/matlab_prog/scripts-and-functions.html
2. MATLAB matrix and array operations
   https://www.mathworks.com/help/matlab/matlab_prog/matrices-and-arrays.html
3. MATLAB mathematical functions
   https://www.mathworks.com/help/matlab/matlab_prog/array-vs-matrix-operations.html
4. MATLAB image processing basics
   https://www.mathworks.com/help/matlab/images.html

Course Material:

5. Course lecture notes on vectorization